

Chuẩn bị môi trường học

Làm **một lần** trước buổi 1, khoảng 20 phút (phần lớn là chờ tải). Sau đó mỗi buổi chỉ cần `python lab.py up`.

Kiểm theo tài liệu chính thức ngày **2026-09-18**. Lệnh cài có thể đổi — lệnh nào không chạy thì mở link nguồn cạnh nó.

Cần gì

Thành phần	Bắt buộc?	Dùng cho
Linux, macOS, hoặc Windows + WSL2	có	cả khoá (khoá được kiểm trên Linux và macOS)
uv — hoặc conda nếu bạn đã quen	có, chọn một	Python 3.12 và thư viện đúng phiên bản của từng buổi
CPU 4 nhân, 16 GB RAM, 30 GB đĩa trống	khuyến nghị	8 GB RAM đủ cho buổi 1-28 và 38-43
API key OpenAQ , EIA (miễn phí)	buổi 10, 41-44, dự án	dữ liệu không khí, điện
Tài khoản Kaggle	buổi 23, đề A	dữ liệu M5 (có bộ dữ phòng mở)
Ollama hoặc llama.cpp	buổi 36-37	chạy LLM mở trên máy, không tốn tiền API
Docker	buổi 41-43	đóng gói pipeline và API

1. Windows: bật WSL2 trước

Cần Windows 10 bản 2004 trở lên hoặc Windows 11. Mở **PowerShell quyền Administrator**, chạy `wsl --install`, khởi động lại máy. Từ đây mọi lệnh của khoá gỗ **trong cửa sổ Ubuntu**. Để thư mục khoá trong `/home/<tên>/...`, không để ở `/mnt/c/...` (đọc ghi chậm hơn nhiều). Máy yếu thì giới hạn RAM cho WSL bằng tệp `%UserProfile%\wslconfig`:

```
[wsl2]
memory=12GB
```

rồi chạy `wsl --shutdown` trong PowerShell.

Nguồn: learn.microsoft.com/windows/wsl/install, [.../wsl/filesystems](#), [.../wsl/wsl-config](#).

2. Chọn cách dựng môi trường: uv hoặc conda

Mỗi buổi chốt đúng phiên bản thư viện đã dùng để ra mọi con số trong tài liệu. Hai cách đều cài đúng các phiên bản đó.

Cách A — uv (khuyến nghị). uv tự tải Python 3.12 và tạo một môi trường riêng cho từng buổi từ tệp khoá phiên bản `uv.lock`, nên máy bạn giống hệt máy tác giả.

```
curl -Lsf https://astral.sh/uv/install.sh | sh      # Linux, macOS, WSL (macOS: brew install uv cũng được)
uv --version                                         # khoá kiểm với uv 0.12.15
```

Cách B — conda (hoặc venv của bạn). Mỗi buổi có `lab/00-nen/requirements.txt`, sinh từ cùng `uv.lock`. Tạo một môi trường Python 3.12, bật nó, rồi dùng `--pip`:

```
conda create -n forecasting python=3.12 && conda activate forecasting
cd buoi-01/lab && python lab.py up --pip
```

Một môi trường conda dùng chung cho nhiều buổi được, nhưng buổi sau có thể cần phiên bản khác (buổi 14–24 dùng pandas 2.3.3 thay vì 3.0.5, vì thư viện dự báo Nixtla chưa chạy với pandas 3). Gặp lỗi phiên bản thì tạo môi trường mới cho buổi đó.

Nguồn: docs.astral.sh/uv/getting-started/installation.

3. Một buổi học vận hành thế nào

```
buoi-07/
├─ BIEU-DO-...-buoi-07.pdf      tài liệu
├─ code/                        điểm xuất phát (có chỗ cố tình sai – bài học là tìm ra nó)
│   └─ lab.ipynb                notebook của Lab
└─ lab/
    ├─ lab.py                   python lab.py up | check | notebook | down
    ├─ 00-nen/                  phiên bản thư viện + danh sách dữ liệu (đừng sửa)
    └─ cham/                    bộ chấm tự động
```

Đứng trong `buoi-07/lab/`:

```
python lab.py up      # dựng môi trường + tải dữ liệu, kiểm sha256 (lần đầu vài phút)
python lab.py notebook # mở JupyterLab ở code/ – mở lab.ipynb, chạy từng ô
python lab.py check   # chấm bài trong code/: đầu buổi ĐỎ, cuối buổi phải XANH
python lab.py down    # xoá môi trường + dữ liệu của buổi khi học xong
```

- **sha256** là "dấu vân tay" của tệp: sai một byte là khác hẳn. `up` kiểm nó để chắc bạn có đúng dữ liệu tác giả dùng.
- Dữ liệu tải về được giữ ở `~/.cache/khoa-forecasting/`: `down` rồi `up` lại không phải tải lại. Đổi chỗ cache: `export KHOA_FORECASTING_CACHE=/o-lon/khoa-forecasting`.
- **VS Code** thay JupyterLab: mở `code/lab.ipynb`, chọn Python `lab/00-nen/.venv` (cách A) hoặc môi trường conda.
- Notebook nạp code từ các tệp `.py` trong `code/`. Bạn sửa **tệp .py** (bộ chấm chấm tệp đó); ô đầu notebook bật tự nạp lại, nên sửa xong chạy lại ô là thấy kết quả mới.

Khi `python lab.py up` báo lỗi

Thông báo	Làm gì
Chưa có uv	cài uv (mục 2), hoặc dùng conda: <code>python lab.py up --pip</code>
sha256 KHÔNG KHỚP	nguồn dữ liệu đã đổi nội dung. Đừng sửa sha256 . Báo giảng viên

Thông báo	Làm gì
thiếu biến môi trường <code>...API_KEY</code>	làm mục 4
Kaggle từ chối (403)	mở link luật cuộc thi trong thông báo, bấm chấp nhận, chạy lại
HTTP 429	nguồn giới hạn tần suất — chờ vài phút rồi chạy lại
mạng chậm, tải dờ	chạy lại <code>python lab.py</code> up : tệp đã tải xong được giữ trong cache

4. API key dữ liệu (miễn phí)

Chỉ cần từ buổi có ghi trong bảng đầu trang. Key **không bao giờ** được commit vào git.

Cách lưu key — chọn một:

- Xuất biến trong `~/.bashrc` (hoặc `~/.zshrc` trên macOS):

```
export OPENAQ_API_KEY="..."
export EIA_API_KEY="..."
```

- Hoặc tạo tệp `.env` trong thư mục `lab/` của buổi (bộ tải dữ liệu tự đọc; biến đã export được ưu tiên hơn `.env`):

```
OPENAQ_API_KEY=...
EIA_API_KEY=...
```

`.env` đã có trong `.gitignore` của khoá.

OpenAQ (chất lượng không khí)

1. Đăng ký tại <https://explore.openaq.org/register>.
2. Lấy key ở <https://explore.openaq.org/account>.
3. Key gửi qua header `X-API-Key` — bộ tải dữ liệu tự làm việc này.

Giới hạn: 60 lượt/phút, 2.000 lượt/giờ. Điều khoản OpenAQ yêu cầu ghi nguồn OpenAQ và tuân theo giấy phép của từng nhà cung cấp dữ liệu. Nguồn: docs.openaq.org.

EIA (tải điện Mỹ)

1. Đăng ký tại <https://www.eia.gov/opendata/register.php> — key gửi về email.
2. Thử nhanh không cần key: EIA có key công khai `DEMO_KEY` (giới hạn rất thấp, chỉ để thử).

Nguồn: eia.gov/opendata/documentation.php.

Kaggle (buổi 23, đề A)

1. Tạo tài khoản, vào <https://www.kaggle.com/settings/api>, tạo token.
2. Đăng nhập **một** trong các cách:

```
uvx kaggle auth login          # đăng nhập qua trình duyệt
export KAGGLE_API_TOKEN="..." # hoặc biến môi trường
# hoặc lưu token vào ~/.kaggle/access_token rồi: chmod 600 ~/.kaggle/access_token
```

Tệp `kaggle.json` kiểu cũ ("Legacy API Credentials") vẫn dùng được.

3. **Mở trang luật cuộc thi và bấm chấp nhận** — ví dụ M5: <https://www.kaggle.com/competitions/m5-forecasting-accuracy/rules>. Chưa chấp nhận thì tải về bị từ chối (HTTP 403).

Luật M5 chỉ cho dùng phi thương mại và **cấm phân phối lại** dữ liệu, nên khoá không kèm dữ liệu M5 — mỗi học viên tự tải bằng tài khoản của mình. Buổi nào dùng M5 cũng có bộ dữ liệu mở dự phòng.

Nguồn: github.com/Kaggle/kaggle-cli — docs.

Hugging Face

Không cần tài khoản để tải dữ liệu và model công khai. Lớp đông người dùng chung một mạng có thể chạm giới hạn tải ẩn danh (3.000 lượt/5 phút/IP) — khi đó tạo token đọc tại <https://huggingface.co/settings/tokens> và `export HF_TOKEN="hf_..."`.

5. LLM chạy trên máy (buổi 36–37)

Buổi 36–37 chạy được **không cần trả tiền API**: dùng model mở chạy local, hoặc bộ chấm dùng bản ghi phản hồi có sẵn. Chọn một trong hai công cụ.

Ollama

```
curl -fsSL https://ollama.com/install.sh | sh      # Linux và WSL (chạy lại lệnh này để cập nhật)
```

macOS: tải ứng dụng từ <https://ollama.com/download> (cần macOS 14 Sonoma trở lên).

```
ollama pull <tên-model>      # tên model cụ thể ghi trong tài liệu buổi 36
ollama run <tên-model>
```

Ollama mở endpoint tương thích OpenAI tại `http://localhost:11434/v1/` (api key điền chuỗi bất kỳ, ví dụ `ollama`). Nguồn: docs.ollama.com, [OpenAI compatibility](#).

llama.cpp

```
brew install llama.cpp      # macOS, Linux có Homebrew
# hoặc tải bản dựng sẵn: https://github.com/ggml-org/llama.cpp/releases
llama-server -m model.gguf -c 4096
```

`llama-server` nghe ở `http://127.0.0.1:8080`, có endpoint `/v1/chat/completions`. Nguồn: [llama.cpp install](#), [server README](#).

6. Docker (buổi 41–43)

Ubuntu / WSL không dùng Docker Desktop: theo docs.docker.com/engine/install/ubuntu (hỗ trợ Ubuntu 22.04, 24.04, 26.04). Cách nhanh cho máy học tập:

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker $USER      # đăng xuất rồi đăng nhập lại
docker compose version
```

Lưu ý bảo mật từ tài liệu Docker: nhóm `docker` "grants root-level privileges to the user".

macOS / Windows: cài Docker Desktop. Trên Windows bật *Settings* → *General* → *Use WSL 2 based engine* và *Resources* → *WSL Integration* cho bản Ubuntu (cần WSL ≥ 2.1.5).

7. Kiểm tra cuối

```
uv --version          # hoặc: conda --version
```

In ra phiên bản là sẵn sàng cho buổi 1.

Phụ lục cho giảng viên / tác giả: công cụ của repo

Chỉ cần khi soạn khoá (sinh PDF, sinh nền buổi, đóng gói), học viên không cần.

```
# thư viện hệ thống cho weasyprint (Ubuntu) + font dùng trong PDF
sudo apt install -y libpango-1.0-0 libharfbuzz0b libpangoft2-1.0-0 libharfbuzz-subset0 \
    fonts-dejavu-core fonts-liberation
# macOS: brew install pango

uv venv .venv && uv pip install --python .venv/bin/python -r tools/requirements.txt
```

Nguồn: [weasyprint — first steps](#). Cách dùng từng công cụ: `CLAUDE.md`, mục "Công cụ".